

SENITY

Senity CLI

Globaler `senity`-Befehl fuer den Senity-Workspace-Container

Version 1.0 — User Guide

Stand Mai 2026

Ziel Onboarding, Installation, Alltagsgebrauch

Tickets #1030 / #1041

Uebersicht

1. Was ist die Senity CLI?
2. Installation
3. Erst-Login
4. Alltagsgebrauch
5. Optionen und Flags
6. Default-Verhalten
7. Konfiguration
8. Parallele Sessions
9. Update
10. Troubleshooting
11. Abgrenzung zum klassischen Workspace

1. Was ist die Senity CLI?

Die Senity CLI stellt einen globalen Befehl `senity` bereit, der den Senity-Workspace-Container ad-hoc startet und das aktuelle Arbeitsverzeichnis als `/workspace/cwd` in den Container mountet. Vorbild sind die CLIs `claude`, `codex` und `gemini`.

Use-Case: irgendwo im Dateisystem `cd projekt-x`, dann `senity` tippen, in der Senity-Claude-Session landen, am Code arbeiten, beim Exit verschwindet der Container, der Ordner bleibt unverändert.

Wann nutzen? Spontane Sessions in fremden Projekt-Ordern, schnelle Demos, Onboarding neuer Teammitglieder. Fuer persistente Multi-Repo-Workspaces mit eigener `.bindings`-Konfiguration bleibt der klassische `claude-senity.sh` / `.bat`-Launcher das richtige Werkzeug.

2. Installation

Linux / macOS

```
curl -fsSL https://git.senity.ai/senity-admin/senity-code/raw/branch/main/senity-cli/ins
```

Installiert nach `~/.local/bin/senity`. Liegt der Pfad nicht im `PATH`, gibt der Installer die noetige Zeile fuer `~/.bashrc` oder `~/.zshrc` aus.

Windows (PowerShell 7+)

```
irm https://git.senity.ai/senity-admin/senity-code/raw/branch/main/senity-cli/install.ps
```

Installiert nach `%USERPROFILE%\senity\bin\senity.ps1` und legt einen `senity.bat`-Shim fuer `cmd.exe` an. Der User-PATH wird automatisch erweitert. Danach eine neue Shell oeffnen.

Voraussetzungen

Komponente	Pruefung	Installation
Docker Desktop / docker-engine	<code>docker info</code>	winget / brew / docs.docker.com
git (fuer Skill-Cache)	<code>git --version</code>	OS-Paketmanager
SSH-Key fuer Skill-Repos	<code>ssh -T git@github.com</code>	siehe Senity-Admin
Bash bzw. PowerShell 7+	<code>bash --version</code> / <code>pwsh --version</code>	OS-Default bzw. winget

3. Erst-Login

```
senity login
```

Fragt nach Senity-Proxy-URL und Proxy-Key, schreibt nach `~/.senity/.env` (Mode 600 auf Unix). Die URL hat den Default `https://sdr.senity.ai/api/claude-proxy`. Den Key liefert der Senity-Admin.

Sicherheit. Der Key landet im Klartext in `~/.senity/.env`. Verzeichnis-Berechtigung ist `0700`, Datei `0600`. Niemals einchecken, niemals weitergeben.

4. Alltagsgebrauch

```
cd ~/projekte/foo
senity
```

Beim ersten Aufruf laeuft ein Auto-Update (Docker-Image pullen, Skills / Commands / Agents / MCPs klonen). Danach startet der Container, mountet `~/projekte/foo` als `/workspace/cwd` und startet die Senity-Claude-Session. Beim Exit ist der Container weg.

Zusatz-Mounts

```
senity --mount ~/.gitconfig:/workspace/.gitconfig:ro
senity --mount /data/shared:/workspace/shared
```

Spezifisches Image-Tag

```
senity --image git.senity.ai/senity-admin/senity-code:1.4.2
```

5. Optionen und Flags

Bash / Shell	PowerShell	Wirkung
<code>--skip-update</code>	<code>-SkipUpdate</code>	Ueberspringt Docker- und Skill-Updates
<code>--no-yolo</code>	<code>-NoYolo</code>	Permission-Prompts aktivieren
<code>--yolo</code>	<code>-Yolo</code>	Yolo explizit an (Default)
<code>--mount H:C[:ro]</code>	<code>-Mount H:C[:ro]</code>	Zusatz-Mount, mehrfach erlaubt
<code>--image <ref></code>	<code>-Image <ref></code>	Image-Tag ueberschreiben
<code>--help</code> / <code>-h</code>	<code>-h</code>	Hilfe
<code>login</code> (Subcommand)	<code>login</code> (Subcommand)	Proxy-Key einrichten

6. Default-Verhalten

Schritt	Default	Skip
Docker-Image pullen	bei jedem Start	<code>--skip-update</code>
Skills / Commands / Agents pullen	bei jedem Start	<code>--skip-update</code>
MCP-Server pullen	bei jedem Start	<code>--skip-update</code>
Yolo-Mode	an	<code>--no-yolo</code>
Workspace-HOME	<code>~/.senity/workspace</code> → <code>/workspace</code>	n/a
cwd-Mount	<code>\$(pwd)</code> → <code>/workspace/cwd</code>	n/a
MCP-User-Config	<code>~/.senity/mcp-config.json</code> (optional)	n/a
SSH-Keys	<code>~/.ssh</code> read-only gemountet	n/a

Update-Default-Philosophie. Auto-Update laeuft bei jedem Aufruf. Das spart das manuelle `senity -update` -Subcommand und garantiert, dass jede Session den aktuellen Skill- und Image-Stand sieht. Power-User koennen mit `--skip-update` bewusst offline arbeiten.

7. Konfiguration

Alle CLI-Daten liegen unter `~/.senity/`:

```
~/.senity/
+-- .env                # SENITY_CHAT_PROXY_URL + KEY (chmod 600)
+-- mcp-config.json     # optional, User-globale MCP-Server
+-- workspace/          # persistentes /workspace (Claude-Settings, Codex-Tokens, ...)
+-- cache/
  +-- skills/           # Klon von murc134/Claude-Skills
  +-- commands/         # Klon von murc134/Claude-Commands
  +-- agents/           # Klon von murc134/Claude-Agents
  +-- senity-mcps/      # Klon von senity/senity-mcps
```

MCP-Server konfigurieren

Eigene MCP-Server oder Token-Overrides legt der User in `~/.senity/mcp-config.json` ab. Format und Beispiel siehe `mcp-config.example.json` im Hauptrepo. Datei wird beim Start read-only in den Container gemountet und mit den Repo-Defaults gemergt.

8. Parallele Sessions

Jeder Aufruf bekommt einen eigenen Container-Namen `senity-<pid>-<epoch>`. Mehrere Terminals koennen ohne Konflikt parallel `senity` aufrufen, jeder bekommt seinen eigenen Container. Beim Exit verschwinden alle (`--rm`).

Der gemeinsame Mount `~/senity/workspace` ist read-write fuer beide Sessions. Vorsicht bei gleichzeitiger Aenderung derselben Settings-Datei, last-write-wins.

9. Update

Die Wrapper-Skripts updaten sich selbst nicht automatisch. Re-Run des Installers ueberschreibt das Script, Cache und `.env` bleiben:

```
# Linux / macOS
curl -fsSL https://git.senity.ai/senity-admin/senity-code/raw/branch/main/senity-cli/install.sh

# Windows
irm https://git.senity.ai/senity-admin/senity-code/raw/branch/main/senity-cli/install.ps1
```

10. Troubleshooting

"Image fehlt und --skip-update verhindert Pull"

Aufruf ohne `--skip-update` wiederholen, der Erstaufruf braucht einen Pull.

"docker pull fehlgeschlagen ... 401 Unauthorized"

Einmalig `docker login git.senity.ai`. Username = Gitea-User, Passwort = Personal Access Token mit Scope `read:package` (Gitea: User Settings -> Applications -> Generate New Token).

"SENITY_CHAT_PROXY_KEY fehlt"

`senity login` ausfuehren.

Skills oder Commands fehlen im Container

Auto-Update braucht SSH-Zugriff auf die Repos. SSH-Agent / Key in `~/ssh/config` pruefen. `--skip-update` nutzt den letzten erfolgreichen Cache-Stand.

Permission-Probleme auf Windows

Falls `senity.ps1` nicht startet: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. Der `.bat`-Shim ruft pwsh mit `-ExecutionPolicy Bypass` auf, fuer cmd.exe ist das transparent.

11. Abgrenzung zum klassischen Workspace

Aspekt	<code>senity</code> (Ad-hoc)	<code>claude-senity</code> (klassisch)
Setup	installer + <code>senity login</code>	Repo klonen + <code>.env</code>
Workspace	cwd des Aufrufs	festes <code>workspace/</code> im Repo
Mounts	einer (cwd) + optionale CLI-Flags	via <code>.bindings</code> -Datei
Projekt-Repos	nicht persistent	persistent unter <code>workspace/projects/</code>
Initial Prompt	keiner	<code>INITIAL_PROMPT.md</code>
Lifecycle	pro Aufruf, <code>--rm</code>	langlebige Sessions
Use-Case	spontan im fremden Ordner	Entwickler-Daily-Workspace